

مقایسه معماری‌های MVVM و MVI در توسعه اپلیکیشن‌های اندروید؛ بررسی ساختار، مدیریت وضعیت و کاربردها

نویسنده: علی محبوب

چکیده

در سال‌های اخیر، توسعه اپلیکیشن‌های اندروید با تغییرات قابل توجهی در حوزه معماری نرم‌افزار همراه بوده است. افزایش پیچیدگی رابط‌های کاربری، گسترش استفاده از برنامه‌نویسی واکنشی (Reactive Programming) و معرفی Jetpack Compose موجب شده است که انتخاب معماری مناسب بیش از گذشته اهمیت پیدا کند. معماری نرم‌افزار نه تنها بر کیفیت کدنویسی تأثیر می‌گذارد، بلکه نقش مهمی در نگهداری، توسعه، مقیاس‌پذیری و تست‌پذیری پروژه ایفا می‌کند.

در میان معماری‌های مختلف، دو الگوی (Model-View-Intent) و (Model-View-ViewModel) (MVVM) بیشترین کاربرد را در توسعه اپلیکیشن‌های مدرن اندروید دارند. معماری MVVM طی سال‌های گذشته به عنوان معماری پیشنهادی گوگل شناخته شده و در بسیاری از پروژه‌های تجاری مورد استفاده قرار گرفته است. در مقابل، معماری MVI با تکیه بر جریان داده یک‌طرفه و مدیریت متمرکز وضعیت برنامه، رویکردی نوین برای توسعه رابط‌های کاربری پیچیده ارائه می‌دهد.

هدف این مقاله، بررسی دقیق ساختار، نحوه عملکرد، مزایا، محدودیت‌ها و کاربردهای هر دو معماری است. همچنین با تحلیل تفاوت‌های اساسی MVVM و MVI، تلاش می‌شود دیدگاهی روشن برای انتخاب معماری مناسب در پروژه‌های اندرویدی ارائه شود.

واژگان کلیدی: اندروید، MVI، MVVM، Jetpack Compose، Kotlin، معماری نرم‌افزار، State Management، معماری رابط کاربری.

۱. مقدمه

با رشد روزافزون صنعت نرم افزارهای موبایل، توسعه دهندگان با چالش‌های متعددی در زمینه طراحی، توسعه و نگهداری پروژه‌های بزرگ روبه‌رو هستند. افزایش تعداد قابلیت‌های نرم افزار، پیچیده‌تر شدن تعاملات کاربر و نیاز به توسعه مداوم محصولات باعث شده است که استفاده از یک معماری مناسب به یکی از مهم‌ترین عوامل موفقیت پروژه تبدیل شود.

در پروژه‌های کوچک، ممکن است ساختار کد در ابتدا ساده و قابل مدیریت باشد، اما با گذشت زمان و اضافه شدن قابلیت‌های جدید، وابستگی میان بخش‌های مختلف برنامه افزایش یافته و نگهداری کد دشوار می‌شود. در چنین شرایطی، نبود یک معماری مناسب موجب افزایش هزینه توسعه، کاهش کیفیت نرم افزار و ایجاد خطاهای متعدد خواهد شد.

معماری نرم افزار مجموعه‌ای از الگوها و قوانین طراحی است که نحوه ارتباط اجزای مختلف سیستم را مشخص می‌کند. هدف اصلی معماری، جداسازی مسئولیت‌ها، کاهش وابستگی بین بخش‌های مختلف برنامه و افزایش قابلیت توسعه و نگهداری نرم افزار است.

در اکوسیستم اندروید، طی سال‌های گذشته معماری‌های متعددی از جمله MVC، MVP، MVVM و MVI معرفی شده‌اند. هر یک از این معماری‌ها با هدف حل بخشی از مشکلات توسعه نرم افزار ارائه شده‌اند و متناسب با نوع پروژه، مزایا و محدودیت‌های خاص خود را دارند.

در میان این معماری‌ها، MVVM به دلیل سادگی، مستندات گسترده و پشتیبانی رسمی توسط Google، به عنوان رایج‌ترین معماری توسعه اندروید شناخته می‌شود. از سوی دیگر، با معرفی Jetpack Compose و گسترش استفاده از StateFlow و Kotlin Coroutines، معماری MVI نیز به دلیل مدیریت یکپارچه وضعیت برنامه و جریان داده یک‌طرفه، توجه بسیاری از توسعه دهندگان را به خود جلب کرده است.

انتخاب میان این دو معماری همواره یکی از موضوعات مورد بحث در میان برنامه‌نویسان اندروید بوده است. برخی پروژه‌ها با استفاده از MVVM عملکرد مطلوبی دارند، در حالی که در پروژه‌های بزرگ و دارای تعاملات پیچیده، استفاده از MVI می‌تواند ساختاری منظم‌تر و قابل پیش‌بینی‌تر ایجاد کند.

در این مقاله تلاش شده است با بررسی ساختار داخلی هر دو معماری، نحوه مدیریت وضعیت برنامه، جریان داده، نحوه پردازش رویدادها و همچنین مزایا و محدودیت‌های هر یک، تصویری جامع از کاربردهای آن‌ها ارائه شود تا توسعه دهندگان بتوانند متناسب با نیاز پروژه، تصمیم آگاهانه‌تری اتخاذ کنند.

۲. مبانی نظری معماری نرم افزار

معماری نرم افزار را می توان نقشه راه توسعه یک سیستم دانست. همان گونه که ساخت یک ساختمان بدون نقشه مهندسی می تواند منجر به مشکلات متعدد شود، توسعه نرم افزار نیز بدون معماری مشخص، به مرور زمان با افزایش پیچیدگی، کاهش خوانایی کد و دشواری در نگهداری مواجه خواهد شد.

یکی از مهم ترین اهداف معماری نرم افزار، جداسازی مسئولیت ها (Separation of Concerns) است. در این رویکرد، هر بخش از سیستم تنها مسئول انجام وظیفه مشخصی است و از انجام مسئولیت های سایر بخش ها خودداری می کند. این اصل باعث می شود تغییرات در یک قسمت، کمترین تأثیر را بر سایر قسمت های برنامه داشته باشد.

علاوه بر جداسازی مسئولیت ها، معماری مناسب باید ویژگی های زیر را نیز فراهم کند:

- خوانایی بالا: ساختار پروژه باید به گونه ای باشد که اعضای تیم بتوانند به سرعت منطق برنامه را درک کنند.
- قابلیت نگهداری: اعمال تغییرات و رفع خطاها بدون ایجاد اثرات جانبی گسترده امکان پذیر باشد.
- تست پذیری: امکان نوشتن آزمون های واحد و آزمون های یکپارچه برای بخش های مختلف برنامه فراهم شود.
- مقیاس پذیری: با افزایش قابلیت های نرم افزار، توسعه سیستم همچنان قابل مدیریت باقی بماند.
- کاهش وابستگی: ارتباط میان لایه های مختلف برنامه تا حد امکان کاهش یابد.

در توسعه اندروید، رعایت این اصول اهمیت ویژه ای دارد؛ زیرا اپلیکیشن های موبایل معمولاً با منابع مختلفی مانند سرویس های وب، پایگاه های داده، حسگرهای دستگاه و رابط های کاربری پویا در ارتباط هستند. مدیریت صحیح این تعاملات بدون استفاده از یک معماری مناسب، در پروژه های متوسط و بزرگ بسیار دشوار خواهد بود.

۳. معماری MVVM (Model–View–ViewModel)

۳-۱. معرفی معماری MVVM

معماری **MVVM (Model-View-ViewModel)** یکی از شناخته‌شده‌ترین الگوهای معماری در توسعه نرم‌افزارهای رابط کاربری است که نخستین‌بار توسط شرکت مایکروسافت برای فناوری WPF معرفی شد. با گذشت زمان، این معماری به دلیل ساختار منظم و قابلیت جداسازی مسئولیت‌ها، در بسیاری از پلتفرم‌ها از جمله اندروید مورد استفاده قرار گرفت.

با معرفی مجموعه کتابخانه‌های **Android Jetpack** و به‌ویژه کلاس **ViewModel**، معماری MVVM به الگوی پیشنهادی گوگل برای توسعه برنامه‌های اندرویدی تبدیل شد. امروزه بسیاری از پروژه‌های تجاری، استارت‌آپ‌ها و حتی محصولات سازمانی بر پایه این معماری توسعه داده می‌شوند.

هدف اصلی MVVM کاهش وابستگی بین رابط کاربری و منطق تجاری برنامه است. در این معماری، هر لایه مسئولیت مشخصی بر عهده دارد و از دسترسی مستقیم به مسئولیت سایر لایه‌ها اجتناب می‌کند.

۲-۳. اجزای اصلی MVVM

معماری MVVM از سه بخش اصلی تشکیل شده است که هر یک وظایف مشخصی را بر عهده دارند.

Model

لایه **Model** مسئول مدیریت داده‌های برنامه است. این داده‌ها ممکن است از منابع مختلفی مانند وب‌سرویس، پایگاه داده محلی یا حافظه موقت دریافت شوند.

به طور معمول، این لایه شامل اجزای زیر است:

- Repository
- Remote Data Source
- Local Data Source
- Room Database
- Ktor Client یا Retrofit
- DataStore

Model هیچ اطلاعی از رابط کاربری ندارد و تنها وظیفه آن تأمین داده و مدیریت منطق مربوط به ذخیره‌سازی اطلاعات است.

View

View مسئول نمایش اطلاعات به کاربر است.

در پروژه‌های جدید اندروید، این بخش معمولاً توسط **Jetpack Compose** پیاده‌سازی می‌شود؛ هرچند در پروژه‌های قدیمی ممکن است از XML استفاده شود.

وظایف View عبارت‌اند از:

- نمایش داده‌ها
- دریافت تعاملات کاربر
- ارسال رویدادها به ViewModel
- مشاهده تغییرات State

نکته مهم این است که View نباید شامل منطق تجاری (Business Logic) باشد.

ViewModel

ViewModel مهم‌ترین بخش این معماری محسوب می‌شود.

این لایه واسطی میان View و Model است و مسئول مدیریت وضعیت برنامه، دریافت داده‌ها و آماده‌سازی آن‌ها برای نمایش در رابط کاربری است.

مهم‌ترین وظایف ViewModel عبارت‌اند از:

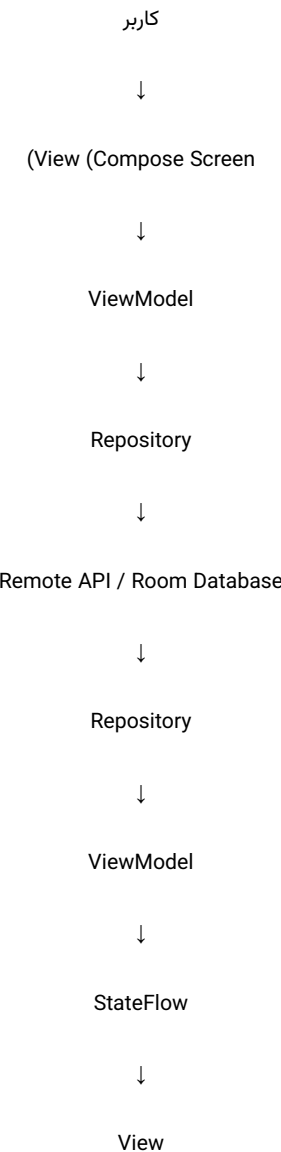
- دریافت داده از Repository
- مدیریت وضعیت صفحه
- مدیریت عملیات غیرهمزمان
- اعتبارسنجی داده‌ها
- تبدیل داده‌ها به فرم قابل نمایش
- مدیریت خطاها

در پروژه‌های مدرن، ViewModel معمولاً با استفاده از **Kotlin Coroutines** و **StateFlow** پیاده‌سازی می‌شود.

۳-۳. جریان داده در MVVM

یکی از ویژگی‌های مهم MVVM، جداسازی مسیر حرکت داده‌ها است.

فرآیند اجرای یک عملیات معمولی را می‌توان به صورت زیر نمایش داد:



در این فرآیند، رابط کاربری تنها وضعیت جدید را مشاهده می‌کند و هرگز به صورت مستقیم با منابع داده ارتباط برقرار نمی‌کند.

۳-۴. نقش Repository در MVVM

در بسیاری از پروژه‌های حرفه‌ای، ViewModel مستقیماً با API یا پایگاه داده ارتباط برقرار نمی‌کند.

این وظیفه به کلاس Repository سپرده می‌شود.

Repository مسئول تصمیم‌گیری درباره منبع داده است.

به عنوان مثال:

- اگر اینترنت در دسترس باشد، اطلاعات از سرور دریافت می‌شود.
- اگر اینترنت در دسترس نباشد، داده‌های ذخیره‌شده در Room نمایش داده می‌شوند.

این موضوع باعث می‌شود ViewModel کاملاً مستقل از نحوه دریافت داده‌ها باشد.

۳-۵. مدیریت وضعیت (State Management)

در گذشته بسیاری از پروژه‌های اندروید از LiveData برای مدیریت وضعیت استفاده می‌کردند؛ اما با معرفی Kotlin Flow و StateFlow، رویکرد توسعه تغییر کرد.

امروزه اغلب پروژه‌های مدرن از ساختاری مشابه نمونه زیر استفاده می‌کنند:

```
data class HomeUiState(
```

```
    val isLoading: Boolean = false
```

```
    ,()val games: List<Game> = emptyList
```

```
val error: String? = null
```

```
)
```

سپس ViewModel این وضعیت را مدیریت می‌کند:

```
private val _uiState = MutableStateFlow(HomeUiState())
```

```
val uiState = _uiState.asStateFlow()
```

در نهایت، رابط کاربری تنها این وضعیت را مشاهده می‌کند.

۳-۶. مزایای MVVM

استفاده از MVVM مزایای متعددی برای توسعه‌دهندگان فراهم می‌کند.

۱. جداسازی مسئولیت‌ها

هر لایه تنها مسئول انجام وظیفه مشخص خود است که این موضوع باعث افزایش خوانایی پروژه می‌شود.

۲. تست‌پذیری مناسب

از آنجا که ViewModel وابستگی مستقیمی به رابط کاربری ندارد، نوشتن Unit Test برای آن بسیار ساده‌تر خواهد بود.

۳. پشتیبانی رسمی گوگل

کتابخانه‌های Android Jetpack مانند:

- ViewModel
- Lifecycle
- LiveData
- SavedStateHandle

همگی با هدف استفاده در MVVM توسعه یافته‌اند.

۴. کاهش وابستگی

رابط کاربری مستقیماً با API یا پایگاه داده ارتباط برقرار نمی‌کند.

۵. هماهنگی مناسب با Compose

اگرچه Compose از MVI استقبال بیشتری می‌کند، اما MVVM نیز با استفاده از StateFlow عملکرد بسیار مناسبی ارائه می‌دهد.

۷-۳. محدودیت‌های MVVM

با وجود مزایای فراوان، MVVM در پروژه‌های بزرگ با چالش‌هایی نیز همراه است.

مدیریت چندین State

در بسیاری از پروژه‌ها برای هر قسمت صفحه State جداگانه‌ای ایجاد می‌شود که در بلندمدت مدیریت آن دشوار خواهد شد.

مدیریت Event ها

رویدادهایی مانند:

- نمایش Snackbar
- Navigation
- Toast
- Dialog

در MVVM معمولاً نیازمند ساختارهای جداگانه مانند SharedFlow یا Channel هستند که مدیریت آن‌ها پیچیدگی بیشتری ایجاد می‌کند.

احتمال ناسازگاری وضعیت‌ها

در صورتی که وضعیت‌های مختلف به صورت مستقل تغییر کنند، ممکن است رابط کاربری با شرایط ناسازگار مواجه شود؛ برای مثال، هم‌زمان مقدار `isLoading = true` و لیست داده‌ها نیز نمایش داده شوند.

افزایش مسئولیت ViewModel

در بسیاری از پروژه‌ها مشاهده می‌شود که ViewModel به مرور زمان حجم زیادی از منطق برنامه را در خود جای می‌دهد و به اصطلاح به یک **God ViewModel** تبدیل می‌شود. این مسئله نگهداری و توسعه پروژه را دشوار می‌کند.

۸-۳. موارد استفاده از MVVM

با توجه به ویژگی‌های این معماری، MVVM در شرایط زیر انتخاب مناسبی است:

- پروژه‌های کوچک و متوسط
- اپلیکیشن‌های سازمانی با منطق تجاری نسبتاً ساده
- تیم‌هایی که تجربه متوسطی در توسعه اندروید دارند
- پروژه‌هایی که نیاز به توسعه سریع دارند
- برنامه‌هایی که تعداد وضعیت‌های رابط کاربری محدود است

در چنین پروژه‌هایی، سادگی پیاده‌سازی و مستندات گسترده MVVM باعث افزایش سرعت توسعه و کاهش پیچیدگی کد خواهد شد.

جمع‌بندی فصل

معماری MVVM با تکیه بر جداسازی مسئولیت‌ها و استفاده از ViewModel به عنوان واسط میان رابط کاربری و لایه داده، یکی از بالغ‌ترین و پراستفاده‌ترین الگوهای معماری در توسعه اندروید محسوب می‌شود. این معماری برای طیف وسیعی از پروژه‌ها عملکرد مطلوبی دارد و به دلیل پشتیبانی رسمی گوگل، همچنان یکی از گزینه‌های اصلی توسعه‌دهندگان است.

با این حال، در پروژه‌هایی که مدیریت وضعیت رابط کاربری پیچیده‌تر می‌شود، محدودیت‌های MVVM بیش از پیش نمایان شده و نیاز به معماری‌هایی با مدیریت متمرکز State، مانند MVI، احساس می‌شود.

۴. معماری (MVI) (Model-View-Intent)

۴-۱. معرفی معماری MVI

با گسترش برنامه‌نویسی واکنشی (Reactive Programming) و افزایش پیچیدگی رابط‌های کاربری، نیاز به معماری‌هایی که بتوانند وضعیت برنامه را به شکلی قابل پیش‌بینی مدیریت کنند، بیش از گذشته احساس شد. یکی از معماری‌هایی که در پاسخ به این نیاز توسعه یافت، (MVI) (Model-View-Intent) است.

MVI بر پایه اصل **Unidirectional Data Flow** یا **جریان داده یک‌طرفه** طراحی شده است. در این معماری تمام تغییرات رابط کاربری تنها از یک مسیر مشخص عبور می‌کنند و هیچ بخشی از سیستم اجازه تغییر مستقیم وضعیت برنامه را ندارد.

این ویژگی موجب می‌شود رفتار برنامه کاملاً قابل پیش‌بینی باشد و فرآیند اشکال‌زدایی و نگهداری پروژه به شکل محسوسی ساده‌تر شود.

با معرفی **Jetpack Compose**، محبوبیت MVI افزایش چشمگیری پیدا کرد؛ زیرا Compose نیز مبتنی بر مفهوم State طراحی شده و هر تغییر در وضعیت برنامه به صورت خودکار باعث بازترسیم رابط کاربری می‌شود.

۴-۲. فلسفه طراحی MVI

برخلاف MVVM که ممکن است چندین وضعیت مستقل در بخش‌های مختلف ViewModel وجود داشته باشد، MVI تمام وضعیت صفحه را در قالب یک شیء واحد مدیریت می‌کند.

در این معماری، وضعیت برنامه همواره از سه مرحله عبور می‌کند:

1. کاربر یک رویداد ایجاد می‌کند.
2. رویداد به Intent تبدیل می‌شود.
3. Intent وضعیت جدید برنامه را تولید می‌کند.

در نتیجه، رابط کاربری هیچ تغییری را مستقیماً اعمال نمی‌کند و تنها وضعیت جدید را نمایش می‌دهد.

این ویژگی باعث می‌شود رفتار برنامه در هر لحظه قابل بازسازی و قابل پیش‌بینی باشد.

۳-۴. اجزای اصلی معماری MVI

معماری MVI معمولاً از چهار بخش اصلی تشکیل می‌شود.

Model

Model همان مسئولیت معماری MVVM را بر عهده دارد.

وظایف Model عبارت‌اند از:

- ارتباط با API
- ارتباط با Room
- مدیریت Repository
- دریافت و ذخیره اطلاعات

Model هیچ وابستگی مستقیمی به رابط کاربری ندارد.

View

View تنها وضعیت جاری را مشاهده کرده و آن را نمایش می‌دهد.

در Jetpack Compose معمولاً این وضعیت از طریق StateFlow یا MutableState دریافت می‌شود.

View هیچ تصمیمی درباره منطق برنامه نمی‌گیرد.

Intent

Intent نماینده تمام تعاملات کاربر است.

به عنوان مثال:

- کلیک روی دکمه ورود
- Refresh صفحه
- انتخاب آیتم
- جستجو
- حذف اطلاعات
- ثبت فرم

در MVI همه تعاملات کاربر ابتدا به Intent تبدیل می‌شوند.

نمونه:

```
sealed interface HomeIntent{
```

```
data object Refresh : HomeIntent
```

```
data class Search(val query:String):HomeIntent
```

```
data class Delete(val id:Int):HomeIntent
```

```
}
```

استفاده از **Sealed Interface** باعث می‌شود تمام رویدادهای صفحه در یک محل مشخص تعریف شوند.

State

مهم‌ترین تفاوت MVI با MVVM در همین قسمت است.

در MVI تمام اطلاعات صفحه داخل یک State واحد قرار می‌گیرند.

نمونه:

```
data class HomeState(  
  
    ,val isLoading:Boolean=false  
  
  
    ,val games:List<Game> = emptyList()  
  
  
    ,val search:String=""  
  
  
    val error:String?=null  
  
  
)
```

تمام رابط کاربری فقط همین State را مشاهده می‌کند.

۴-۴. جریان داده در MVI

جریان داده در MVI همیشه یک مسیر ثابت دارد.

کاربر



مشاهده می‌شود که هیچ بخشی خارج از این مسیر اجازه تغییر State را ندارد.

۴-۵. مفهوم Reducer

Reducer قلب معماری MVI محسوب می‌شود.

Reducer مسئول تولید وضعیت جدید برنامه است.

به عبارت دیگر:

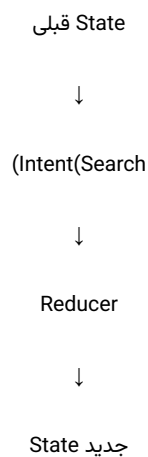
Reducer همیشه دو ورودی دارد:

- State فعلی
- Intent جدید

و خروجی آن:

State جدید است.

برای مثال:



در بسیاری از کتابخانه‌های MVI مانند Orbit، Mavericks یا Redux این مفهوم به‌وضوح مشاهده می‌شود.

۴-۶. Single Source of Truth

یکی از مهم‌ترین مفاهیم MVI اصل **Single Source of Truth** است.

این اصل بیان می‌کند که:

تمام رابط کاربری باید تنها از یک منبع واحد وضعیت خود را دریافت کند.

به بیان ساده‌تر:

هیچ متغیر جداگانه‌ای برای نمایش وضعیت صفحه وجود ندارد.

تمام اطلاعات شامل:

- Loading
- لیست اطلاعات
- متن جستجو
- خطا
- وضعیت Dialog
- وضعیت BottomSheet
- SnackBar
- Navigation

همگی داخل یک State قرار می‌گیرند.

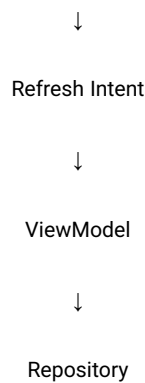
این ویژگی باعث می‌شود احتمال ناسازگاری رابط کاربری تقریباً از بین برود.

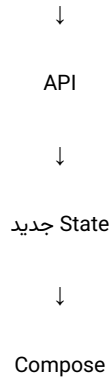
۷-۴. مدیریت رویدادها

در MVI رویدادها نیز ساختار مشخصی دارند.

برای مثال:

کاربر روی دکمه Refresh کلیک می‌کند.





در نتیجه تمام مسیر اجرای برنامه قابل مشاهده خواهد بود.

۴-۸. مدیریت State در Jetpack Compose

یکی از دلایل محبوبیت MVI، هماهنگی بسیار بالای آن با Compose است.

در Compose هر زمان State تغییر کند، رابط کاربری مجدداً رسم می‌شود.

به همین دلیل در بسیاری از پروژه‌ها تنها کافی است نوشته شود:

```
val state by viewModel.state.collectAsState()
```

سپس تمام صفحه فقط از همین State استفاده می‌کند.

۴-۹. مزایای MVI

معماری MVI مزایای متعددی دارد که آن را به گزینه‌ای مناسب برای پروژه‌های بزرگ تبدیل کرده است.

جریان داده قابل پیش‌بینی

تمام تغییرات از یک مسیر عبور می‌کنند و رفتار برنامه کاملاً مشخص است.

مدیریت ساده State

به جای چندین متغیر، تنها یک State مدیریت می‌شود.

تست‌پذیری بسیار بالا

از آنجا که Reducer فقط State جدید تولید می‌کند، نوشتن Unit Test بسیار ساده خواهد بود.

هماهنگی کامل با Compose

Compose دقیقاً براساس State طراحی شده است.

به همین دلیل MVI عملکرد بسیار مناسبی در این فریم‌ورک دارد.

کاهش خطاهای رابط کاربری

وجود تنها یک State باعث می‌شود احتمال نمایش اطلاعات ناسازگار تقریباً از بین برود.

مناسب برای پروژه‌های بزرگ

در پروژه‌هایی با صفحات متعدد، فرم‌های پیچیده و تعاملات زیاد کاربر، MVI ساختاری بسیار منظم ایجاد می‌کند.

۱۰-۴. محدودیت‌های MVI

با وجود مزایای فراوان، این معماری بدون نقص نیست.

یادگیری دشوارتر

درک مفاهیمی مانند Reducer، Intent، و جریان داده یک‌طرفه برای توسعه‌دهندگان مبتدی ممکن است زمان‌بر باشد.

افزایش حجم کدنویسی

در مقایسه با MVVM، برای هر صفحه معمولاً باید کلاس‌های جداگانه‌ای برای Intent، State، و گاهی Effect تعریف شود.

پیچیدگی در پروژه‌های کوچک

برای پروژه‌های ساده، استفاده از MVI ممکن است منجر به افزایش غیرضروری پیچیدگی و زمان توسعه شود.

نیاز به انضباط تیمی

اجرای صحیح MVI مستلزم پایبندی اعضای تیم به اصول معماری است. هرگونه تغییر مستقیم در State یا دور زدن جریان داده، فلسفه این معماری را نقض می‌کند.

جمع‌بندی فصل

MVI معماری‌ای است که با هدف مدیریت متمرکز وضعیت برنامه و ایجاد جریان داده قابل پیش‌بینی طراحی شده است. استفاده از یک State واحد، کاهش ناسازگاری‌های رابط کاربری و هماهنگی بالا با Jetpack Compose، این معماری را به گزینه‌ای مناسب برای پروژه‌های بزرگ و پیچیده تبدیل کرده است.

با این حال، هزینه اولیه یادگیری و حجم بیشتر کدنویسی باعث می‌شود MVI همیشه بهترین انتخاب نباشد. انتخاب این معماری باید با توجه به نیازهای پروژه، اندازه تیم توسعه و پیچیدگی منطق کسب‌وکار انجام شود.

۵. تحلیل تطبیقی معماری‌های MVVM و MVI

۵-۱. مقدمه

معماری‌های MVVM و MVI هر دو با هدف ایجاد ساختاری منظم، قابل توسعه و قابل نگهداری برای نرم‌افزار طراحی شده‌اند، اما رویکرد آن‌ها در مدیریت وضعیت برنامه، پردازش رویدادها و تعامل با رابط کاربری تفاوت‌های بنیادینی دارد.

در نگاه اول ممکن است این دو معماری بسیار مشابه به نظر برسند؛ زیرا در هر دو، **View** از طریق **ViewModel** با لایه داده ارتباط برقرار می‌کند. با این حال، تفاوت اصلی در نحوه مدیریت **State** و جریان داده (**Data Flow**) است. این تفاوت در پروژه‌های بزرگ، اثر مستقیمی بر کیفیت کد، قابلیت توسعه و تجربه توسعه‌دهندگان خواهد داشت.

در این فصل، این دو معماری از جنبه‌های مختلف مورد تحلیل قرار می‌گیرند.

۵-۲. مدیریت وضعیت (State Management)

مدیریت وضعیت یکی از مهم‌ترین معیارهای ارزیابی معماری‌های رابط کاربری است.

در MVVM، وضعیت صفحه معمولاً از چندین متغیر مستقل تشکیل می‌شود.

به عنوان نمونه:

```
val users: List<User>
```

```
val isLoading: Boolean
```

```
?val error: String
```

```
?val selectedUser: User
```

```
val searchText: String
```

هر یک از این متغیرها می‌توانند به صورت مستقل تغییر کنند. اگرچه این روش در پروژه‌های کوچک ساده و قابل فهم است، اما با افزایش پیچیدگی صفحات، احتمال ایجاد ناسازگاری میان وضعیت‌ها افزایش می‌یابد.

برای مثال، ممکن است مقدار `isLoading` همچنان برابر با `true` باشد، در حالی که داده‌ها قبلاً دریافت و نمایش داده شده‌اند. چنین شرایطی می‌تواند به رفتارهای پیش‌بینی‌نشده در رابط کاربری منجر شود.

در مقابل، MVI از یک **State واحد** استفاده می‌کند که نمایانگر وضعیت کامل صفحه است.

```
data class HomeState(
```

```
,val isLoading: Boolean
```

```
,val users: List<User
```

```
,val searchQuery: String
```

```
,?val selectedUser: User
```

```
?val error: String
```

```
)
```

در این رویکرد، هر تغییر منجر به تولید یک نسخه جدید از State می‌شود و رابط کاربری همواره بر اساس همین وضعیت واحد به‌روزرسانی خواهد شد.

۳-۵. جریان داده (Data Flow)

یکی از اساسی‌ترین تفاوت‌های MVVM و MVI، نحوه حرکت داده‌ها در سیستم است.

در MVVM، اگرچه معمولاً جریان داده از ViewModel به View یک‌طرفه است، اما در عمل رویدادهای مختلف ممکن است از مسیرهای متفاوتی پردازش شوند. برای مثال، مدیریت ناوبری، نمایش پیام‌های موقت (Snackbar) یا رویدادهای یک‌بارمصرف اغلب از مکانیزم‌هایی مانند `SharedFlow` یا `Channel` استفاده می‌کند. این موضوع می‌تواند در پروژه‌های بزرگ، مسیرهای مختلفی برای انتقال اطلاعات ایجاد کند.

در MVI، تمام تغییرات تنها از یک مسیر مشخص عبور می‌کنند:

User Action

|



Intent

|



ViewModel

|



Reducer

|



New State

|



View

وجود این مسیر ثابت باعث می‌شود رفتار برنامه کاملاً قابل پیش‌بینی باشد و توسعه‌دهندگان بتوانند روند تغییرات را به‌سادگی دنبال کنند.



۴-۵. مدیریت رویدادها (Event Handling)

در بسیاری از اپلیکیشن‌ها، رویدادهایی مانند موارد زیر به‌طور مداوم رخ می‌دهند:

- کلیک روی دکمه‌ها
- جستجو
- حذف اطلاعات
- ناوبری بین صفحات
- نمایش پیام‌های موقت
- نمایش Dialog

در MVVM، معمولاً هر یک از این رویدادها به‌صورت جداگانه مدیریت می‌شوند. این موضوع اگرچه انعطاف‌پذیری بالایی ایجاد می‌کند، اما ممکن است موجب افزایش پراکندگی منطق برنامه شود.

در مقابل، MVI تمام تعاملات کاربر را در قالب **Intent** مدل‌سازی می‌کند. این رویکرد مزایای متعددی دارد:

- تمامی تعاملات در یک نقطه متمرکز می‌شوند.
- امکان ثبت و بررسی تاریخچه رویدادها فراهم است.
- تست رفتار سیستم ساده‌تر می‌شود.
- احتمال فراموش شدن یک مسیر اجرایی کاهش می‌یابد.

۵-۵. پیچیدگی کد

یکی از نکات مهم در انتخاب معماری، میزان پیچیدگی پیاده‌سازی آن است.

در پروژه‌های کوچک، MVVM معمولاً نیازمند کلاس‌ها و اجزای کمتری است و توسعه‌دهنده می‌تواند با سرعت بیشتری قابلیت‌های جدید را پیاده‌سازی کند.

در مقابل، MVI برای هر صفحه معمولاً شامل اجزای زیر است:

- State
- Intent

- ViewModel
- Reducer
- Effect (در صورت نیاز)

اگرچه این ساختار در ابتدا حجم کد را افزایش می‌دهد، اما در پروژه‌های بزرگ موجب انسجام بیشتر و کاهش پیچیدگی در بلندمدت خواهد شد.

۵-۶. تست‌پذیری (Testability)

یکی از معیارهای مهم در معماری نرم‌افزار، قابلیت تست واحد (Unit Testing) است.

در MVVM، منطق اصلی برنامه در ViewModel قرار دارد و می‌توان رفتار آن را با استفاده از آزمون‌های واحد بررسی کرد. با این حال، زمانی که ViewModel شامل تعداد زیادی وضعیت و رویداد شود، نوشتن آزمون‌ها نیز پیچیده‌تر خواهد شد.

در MVI، به دلیل وجود Reducer و جریان داده مشخص، فرآیند تست ساده‌تر است. کافی است State اولیه و یک Intent مشخص را به Reducer ارسال کنیم و سپس State خروجی را با مقدار مورد انتظار مقایسه کنیم. این ویژگی باعث می‌شود تست رفتار برنامه مستقل از رابط کاربری انجام شود.

۵-۷. عملکرد (Performance)

از نظر عملکرد، تفاوت محسوسی میان MVVM و MVI وجود ندارد؛ زیرا هر دو معماری می‌توانند از فناوری‌هایی مانند Flow، Kotlin Coroutines و StateFlow استفاده کنند.

با این حال، در MVI به دلیل ایجاد نسخه جدیدی از State در هر تغییر، ممکن است نگرانی‌هایی درباره مصرف حافظه ایجاد شود. در عمل، با توجه به استفاده از **Immutable Data** و بهینه‌سازی‌های انجام‌شده در Kotlin و Jetpack Compose، این هزینه معمولاً ناچیز است و تأثیر محسوسی بر عملکرد برنامه ندارد.

در مقابل، مزیت بزرگ MVI در کاهش خطاهای منطقی و افزایش قابلیت پیش‌بینی رفتار برنامه است که در پروژه‌های بزرگ اهمیت بیشتری نسبت به هزینه اندک ایجاد اشیای جدید دارد.

۸-۵. مقیاس‌پذیری (Scalability)

یکی از مهم‌ترین معیارهای انتخاب معماری، توانایی آن در پشتیبانی از رشد پروژه است.

در پروژه‌های کوچک و متوسط، MVVM به دلیل سادگی، گزینه‌ای مناسب محسوب می‌شود و معمولاً پاسخگوی نیازهای پروژه خواهد بود.

اما با افزایش تعداد صفحات، پیچیده‌تر شدن تعاملات کاربر و رشد تیم توسعه، مدیریت وضعیت‌های متعدد در MVVM می‌تواند دشوار شود.

در چنین شرایطی، MVI با استفاده از State واحد، جریان داده یک‌طرفه و ساختار منظم، نگهداری و توسعه پروژه را ساده‌تر می‌کند.

به همین دلیل، بسیاری از شرکت‌های بزرگ در پروژه‌هایی با رابط کاربری پیچیده از معماری‌هایی مبتنی بر اصول MVI یا Redux استفاده می‌کنند.

جمع‌بندی فصل

تحلیل انجام‌شده نشان می‌دهد که هیچ‌یک از دو معماری MVVM و MVI را نمی‌توان به صورت مطلق برتر دانست. انتخاب معماری باید بر اساس عواملی مانند اندازه پروژه، پیچیدگی منطق کسب‌وکار، تجربه تیم توسعه، الزامات نگهداری و برنامه توسعه آینده انجام شود.

MVVM به دلیل سادگی، مستندات گسترده و پشتیبانی رسمی، همچنان انتخابی مناسب برای بسیاری از پروژه‌ها است. در مقابل، MVI با ارائه مدیریت متمرکز وضعیت و جریان داده قابل پیش‌بینی، گزینه‌ای قدرتمند برای پروژه‌های بزرگ و رابط‌های کاربری پیچیده به شمار می‌رود.

۶. تجربه استفاده از MVVM و MVI در پروژه‌های واقعی اندروید

۶-۱. مقدمه

انتخاب یک معماری مناسب تنها بر اساس مقایسه ویژگی‌های فنی آن امکان‌پذیر نیست. بسیاری از تفاوت‌های واقعی معماری‌ها در طول چرخه توسعه نرم‌افزار و هنگام نگهداری پروژه آشکار می‌شوند. ممکن است معماری‌ای که در ابتدای پروژه ساده و کارآمد به نظر می‌رسد، پس از اضافه شدن قابلیت‌های متعدد، توسعه تیم و افزایش تعاملات کاربر با چالش‌هایی روبه‌رو شود.

به همین دلیل، بررسی معماری‌ها از منظر تجربه عملی و شرایط واقعی پروژه، مکمل مناسبی برای تحلیل‌های نظری محسوب می‌شود.

۶-۲. MVVM در پروژه‌های کوچک و متوسط

در پروژه‌هایی که دامنه مشخصی دارند و پیچیدگی رابط کاربری محدود است، MVVM معمولاً انتخابی مناسب به شمار می‌رود. ساختار ساده این معماری، منحنی یادگیری کوتاه و پشتیبانی کامل ابزارهای اندروید موجب می‌شود تیم توسعه بتواند در مدت زمان کوتاهی به بهره‌وری مطلوب برسد.

برای مثال، در اپلیکیشن‌هایی مانند:

- نمایش اخبار
- فروشگاه‌های کوچک
- برنامه‌های آموزشی
- نمایش اطلاعات کاربران
- اپلیکیشن‌های نمونه (MVP)

استفاده از MVVM نه تنها نیازهای پروژه را برآورده می‌کند، بلکه باعث کاهش زمان توسعه و نگهداری نیز می‌شود.

در چنین پروژه‌هایی معمولاً هر صفحه دارای یک ViewModel مستقل است و مدیریت وضعیت پیچیدگی زیادی ندارد.

۳-۶. چالش‌های MVVM در پروژه‌های بزرگ

با افزایش ابعاد پروژه، مسئولیت ViewModel نیز به مرور بیشتر می‌شود. در بسیاری از پروژه‌های سازمانی مشاهده می‌شود که ViewModel علاوه بر مدیریت وضعیت، وظایف دیگری مانند اعتبارسنجی داده‌ها، مدیریت نوبری، هماهنگی میان چند منبع داده و حتی بخشی از منطق تجاری را نیز بر عهده می‌گیرد.

در نتیجه، کلاس ViewModel ممکن است به صدها یا حتی هزاران خط کد برسد که این مسئله خوانایی، تست‌پذیری و نگهداری پروژه را کاهش می‌دهد.

از سوی دیگر، مدیریت رویدادهای موقتی مانند نمایش پیام، باز کردن پنجره‌های محاوره‌ای یا انجام عملیات نوبری معمولاً نیازمند استفاده از سازوکارهای جداگانه‌ای نظیر `Channel` یا `SharedFlow` است. این موضوع اگرچه قابل مدیریت است، اما با افزایش تعداد صفحات می‌تواند ساختار پروژه را پیچیده‌تر کند.

۴-۶. MVI در پروژه‌های پیچیده

در پروژه‌هایی که شامل فرم‌های طولانی، تعاملات هم‌زمان، بارگذاری تدریجی اطلاعات، فیلترهای متعدد یا به‌روزرسانی مداوم رابط کاربری هستند، MVI می‌تواند ساختاری منظم‌تر ارائه دهد.

یکی از مهم‌ترین مزایای این معماری، وجود یک مسیر مشخص برای تغییر وضعیت برنامه است. هر تغییر ابتدا به Intent تبدیل می‌شود، سپس در ViewModel پردازش شده و در نهایت از طریق Reducer وضعیت جدید تولید می‌شود. این روند باعث می‌شود منشأ هر تغییر به راحتی قابل ردیابی باشد.

در پروژه‌های بزرگ که چندین توسعه‌دهنده به صورت هم‌زمان روی یک صفحه کار می‌کنند، چنین ساختاری از ایجاد تغییرات ناخواسته جلوگیری کرده و هماهنگی میان اعضای تیم را افزایش می‌دهد.

۵-۶. نقش Jetpack Compose در انتخاب معماری

معرفی Jetpack Compose نقطه عطفی در توسعه رابط کاربری اندروید محسوب می‌شود. برخلاف سیستم سنتی مبتنی بر XML، Compose بر پایه مفهوم **State** طراحی شده است و هر زمان وضعیت تغییر کند، رابط کاربری به صورت خودکار بازترسیم می‌شود.

این ویژگی موجب شده است که معماری‌هایی با جریان داده یک‌طرفه، مانند MVI، هماهنگی بیشتری با Compose داشته باشند.

با این حال، MVVM نیز با استفاده از **StateFlow** یا **MutableState** عملکرد بسیار مطلوبی ارائه می‌دهد و همچنان انتخاب اصلی بسیاری از تیم‌های توسعه است.

در عمل، انتخاب معماری بیشتر به نحوه مدیریت وضعیت و پیچیدگی پروژه وابسته است تا نوع ابزار طراحی رابط کاربری.

۶-۶. اشتباهات رایج در پیاده‌سازی MVVM

در بسیاری از پروژه‌ها، مشکلات موجود به دلیل ضعف معماری نیست، بلکه ناشی از پیاده‌سازی نادرست آن است. برخی از رایج‌ترین اشتباهات عبارت‌اند از:

- قرار دادن منطق تجاری در View
- برقراری ارتباط مستقیم View با Repository
- تبدیل ViewModel به محلی برای انجام تمام وظایف برنامه
- استفاده هم‌زمان از چندین منبع مستقل برای مدیریت وضعیت
- وابسته کردن ViewModel به کلاس‌های رابط کاربری

چنین مواردی باعث می‌شوند مزایای MVVM از بین برود و ساختار پروژه به مرور زمان دچار آشفتگی شود.

۶-۷. اشتباهات رایج در پیاده‌سازی MVI

در MVI نیز رعایت نکردن اصول معماری می‌تواند مشکلات متعددی ایجاد کند. از جمله:

- تغییر مستقیم State به جای تولید نسخه جدید
- تعریف State‌های بسیار بزرگ و غیرضروری
- استفاده از Intent برای عملیات داخلی که ارتباطی با تعامل کاربر ندارند
- بی‌توجهی به تفکیک مسئولیت‌ها و انتقال منطق تجاری به ViewModel

این اشتباهات علاوه بر افزایش پیچیدگی، فلسفه اصلی MVI را که مبتنی بر جریان داده یک‌طرفه است، تضعیف می‌کنند.

۶-۸. معیارهای انتخاب معماری

انتخاب میان MVVM و MVI نباید بر اساس محبوبیت یا ترندهای روز انجام شود، بلکه باید متناسب با ویژگی‌های پروژه باشد. مهم‌ترین معیارهای تصمیم‌گیری عبارت‌اند از:

- **اندازه پروژه:** در پروژه‌های کوچک، MVVM معمولاً کافی است؛ در پروژه‌های بزرگ، MVI مزایای بیشتری ارائه می‌دهد.
 - **پیچیدگی رابط کاربری:** هرچه تعاملات کاربر و وضعیت‌های صفحه بیشتر باشد، مدیریت متمرکز State اهمیت بیشتری پیدا می‌کند.
 - **تجربه تیم توسعه:** تیم‌هایی که با مفاهیم جریان داده یک‌طرفه و مدیریت وضعیت آشنا هستند، می‌توانند از مزایای MVI بهره بیشتری ببرند.
 - **نیاز به تست‌پذیری:** اگر آزمون‌های واحد و یکپارچه بخش مهمی از فرآیند توسعه باشند، ساختار منظم MVI می‌تواند نگهداری آزمون‌ها را ساده‌تر کند.
 - **برنامه توسعه آینده:** اگر انتظار می‌رود پروژه در آینده قابلیت‌های متعددی دریافت کند، انتخاب معماری مقیاس‌پذیر از ابتدا می‌تواند هزینه‌های توسعه را کاهش دهد.
-

جمع‌بندی فصل

تجربه پروژه‌های واقعی نشان می‌دهد که موفقیت یک معماری بیش از آنکه به نام آن وابسته باشد، به نحوه پیاده‌سازی و میزان پایبندی تیم توسعه به اصول طراحی بستگی دارد. MVVM و MVI هر دو در صورت اجرای صحیح می‌توانند ساختاری پایدار و قابل نگهداری ایجاد کنند.

در پروژه‌هایی که سادگی، سرعت توسعه و منابع آموزشی اهمیت بیشتری دارند، MVVM انتخاب مناسبی است. در مقابل، زمانی که پروژه با مدیریت پیچیده وضعیت، تعاملات گسترده کاربر و نیاز به مقیاس‌پذیری بالا مواجه است، MVI می‌تواند مزایای قابل توجهی ارائه دهد.

۷. نتیجه‌گیری و پیشنهادها

۷-۱. نتیجه‌گیری

انتخاب معماری مناسب یکی از تصمیمات راهبردی در فرآیند توسعه نرم‌افزار است که تأثیر مستقیمی بر کیفیت کد، قابلیت نگهداری، توسعه‌پذیری و هزینه‌های آتی پروژه دارد. با رشد روزافزون پیچیدگی اپلیکیشن‌های موبایل و افزایش تعاملات کاربر، اهمیت مدیریت صحیح وضعیت (State Management) و تفکیک مسئولیت‌ها بیش از گذشته مورد توجه قرار گرفته است.

در این مقاله، دو معماری پرکاربرد (MVVM) (Model-View-ViewModel) و (MVI) (Model-View-Intent) از جنبه‌های مختلف مورد بررسی قرار گرفتند. نتایج این بررسی نشان می‌دهد که هر دو معماری بر پایه اصل جداسازی مسئولیت‌ها طراحی شده‌اند، اما در نحوه مدیریت وضعیت، جریان داده و پردازش تعاملات کاربر تفاوت‌های قابل توجهی دارند.

معماری MVVM با ساختاری ساده، مستندات گسترده و پشتیبانی رسمی از سوی اکوسیستم Android Jetpack، همچنان یکی از رایج‌ترین گزینه‌ها برای توسعه اپلیکیشن‌های اندروید محسوب می‌شود. این معماری برای پروژه‌هایی با پیچیدگی کم تا متوسط، عملکرد مناسبی ارائه می‌دهد و با استفاده از ابزارهایی مانند ViewModel، StateFlow و Repository می‌تواند ساختاری منظم و قابل نگهداری ایجاد کند.

در مقابل، معماری MVI با تکیه بر مدیریت متمرکز وضعیت و جریان داده یک‌طرفه، راهکاری مناسب برای پروژه‌هایی است که دارای رابط کاربری پیچیده، تعاملات متعدد و نیاز به پیش‌بینی‌پذیری بالای رفتار سیستم هستند. استفاده از یک State واحد، مدل‌سازی تعاملات کاربر به‌صورت Intent و تولید وضعیت جدید از طریق Reducer باعث می‌شود ساختار برنامه شفاف‌تر، تست‌پذیرتر و مقیاس‌پذیرتر باشد.

با وجود این، استفاده از MVI همواره بهترین انتخاب نیست. در پروژه‌های کوچک، هزینه اولیه پیاده‌سازی، افزایش حجم کد و نیاز به آشنایی عمیق‌تر تیم توسعه با مفاهیم معماری می‌تواند موجب کاهش بهره‌وری شود. بنابراین انتخاب این معماری باید بر اساس نیازهای واقعی پروژه و نه صرفاً محبوبیت آن انجام شود.

از سوی دیگر، استفاده از MVVM نیز به معنای چشم‌پوشی از اصول مدیریت صحیح وضعیت نیست. در صورتی که ViewModel‌ها بیش از اندازه بزرگ شوند یا مدیریت رویدادها و وضعیت‌ها به‌صورت پراکنده انجام گیرد، ساختار پروژه به مرور زمان پیچیده و نگهداری آن دشوار خواهد شد.

بر این اساس می‌توان نتیجه گرفت که هیچ معماری را نمی‌توان به‌عنوان بهترین راهکار برای تمام پروژه‌ها معرفی کرد. انتخاب معماری باید با در نظر گرفتن عواملی نظیر اندازه پروژه، پیچیدگی منطق کسب‌وکار، تجربه تیم توسعه، الزامات نگهداری و برنامه توسعه آینده انجام شود.

۷-۲. پیشنهادها

با توجه به بررسی‌های انجام‌شده، پیشنهادهای زیر برای توسعه‌دهندگان اندروید ارائه می‌شود:

1. از انتخاب معماری صرفاً بر اساس محبوبیت یا ترندهای روز خودداری کنید. هر معماری باید متناسب با نیازهای واقعی پروژه انتخاب شود.
 2. در پروژه‌های کوچک و متوسط، MVVM معمولاً انتخاب مناسبی است. این معماری با ابزارهای رسمی Android Jetpack سازگاری کامل دارد و فرآیند توسعه را ساده‌تر می‌کند.
 3. در پروژه‌های بزرگ یا دارای رابط کاربری پیچیده، MVI می‌تواند ساختار منظم‌تر و قابل پیش‌بینی‌تری ارائه دهد. استفاده از یک State واحد و جریان داده یک‌طرفه، مدیریت پروژه را در بلندمدت تسهیل می‌کند.
 4. از بزرگ شدن بیش از حد ViewModel جلوگیری کنید. حتی در معماری MVVM نیز لازم است منطق تجاری، مدیریت داده و مسئولیت‌های مختلف به لایه‌های مناسب منتقل شوند.
 5. از Immutable State استفاده کنید. تغییرناپذیر بودن وضعیت برنامه، احتمال بروز خطاهای ناشی از تغییرات ناخواسته را کاهش می‌دهد.
 6. پیش از انتخاب معماری، الزامات نگهداری و توسعه آینده پروژه را ارزیابی کنید. معماری مناسب امروز باید پاسخگوی نیازهای فردای پروژه نیز باشد.
-

جدول راهنمای انتخاب معماری

MVI	MVVM	معیار
★★	★★★★★★	پروژه کوچک
★★★★	★★★★★★	پروژه متوسط
★★★★★★	★★★★	پروژه بزرگ
★★★	★★★★★★	رابط کاربری ساده
★★★★★★	★★★	رابط کاربری پیچیده
★★★★★★	★★★	مدیریت State
★★★	★★★★★★	یادگیری
★★★★★★	★★★★	تست پذیری
★★★★★★	★★★★	مقیاس پذیری



هماهنگی با Jetpack
Compose



سخن پایانی

توسعه نرم افزار بیش از آنکه به انتخاب یک الگوی معماری وابسته باشد، به درک صحیح اصول مهندسی نرم افزار و پایبندی به آن‌ها وابسته است. معماری، ابزاری برای سازمان‌دهی کد و مدیریت پیچیدگی است، نه هدف نهایی توسعه.

تجربه نشان می‌دهد تیم‌هایی که اصولی مانند تفکیک مسئولیت‌ها (Separation of Concerns)، کاهش وابستگی (Loose Coupling)، تک‌مسئولیتی (Single Responsibility) و مدیریت صحیح وضعیت را رعایت می‌کنند، صرف نظر از انتخاب MVVM یا MVI، نرم‌افزارهایی پایدارتر، قابل توسعه‌تر و باکیفیت‌تر تولید خواهند کرد.

در نهایت، انتخاب میان MVVM و MVI نباید به یک تصمیم مطلق تبدیل شود؛ بلکه باید بر پایه تحلیل نیازهای پروژه، توانمندی تیم توسعه و اهداف بلندمدت محصول انجام گیرد. یک توسعه‌دهنده حرفه‌ای، علاوه بر تسلط بر هر دو معماری، باید بتواند متناسب با شرایط، مناسب‌ترین الگو را انتخاب و به‌درستی پیاده‌سازی کند.

منابع (References)

1. Android Developers. *Guide to App Architecture*. Google
2. Android Developers. *State and Jetpack Compose*. Google
3. Android Developers. *ViewModel Overview*. Google
4. Kotlin Documentation. *Coroutines and Flow*. JetBrains
5. JetBrains. *Kotlin Language Documentation*
6. Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002
7. Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017
8. Eric Evans. *Domain-Driven Design*. Addison-Wesley, 2003

پایان

